Лабораторийн ажил №3

Процесс ба санах ой — Бүлэг 3, Practice Exercises

F.CSM302 Үйлдлийн систем
Гүйцэтгэсэн: Б.Билгүүн (B190910014)

Сурах бичиг: A. Silberschatz, P. B. Galvin, G. Gagne, Operating System Concepts, 10th ed., Wiley, 2018

Дасгал 3.1

Асуулт. Зураг 3.30-д үзүүлсэн програмыг ашиглан LINE A мөрөнд ямар гаралт гарахыг тайлбарла.

Хариулт.

```c
int value = 5;
int main() {
    pid_t pid = fork();
    if (pid == 0) {          /* child process */
        value += 15;
        return 0;
    }
    else if (pid > 0) {      /* parent process */
        wait(NULL);
        printf("PARENT: value = %d", value);   /* LINE A */
        return 0;
    }
}
```

LINE A дээр `PARENT: value = 5` гэж хэвлэгдэнэ.

`fork()` дуудагдахад хүү процесс эцэг процессын хаягийн орны (address space) тусдаа хуулбарыг авна. Хүү процесс өөрийн хуулбар дахь `value`-г 20 болгосон ч энэ өөрчлөлт эцэг процессын санах ойд нөлөөлөхгүй. Эцэг процесс `wait()`-ээр хүүгээ дуустал хүлээгээд өөрийн хуулбарыг хэвлэдэг тул утга нь 5 хэвээр байна.

Шалгалт (Ubuntu 22.04, gcc 11.4):

```
$ ./fig3_30
PARENT: value = 5
```

Дасгал 3.2

Асуулт. Эхний эцэг процессыг оруулаад Зураг 3.31-д үзүүлсэн програм хэдэн процесс үүсгэх вэ?

Хариулт.

```c
int main() {
    fork();   /* fork a child process */
    fork();   /* fork another child process */
    fork();   /* and fork another */
    return 0;
}
```

Нийт **8 процесс** үүснэ (эхний эцэг процессыг оруулаад).

`fork()` бүр тухайн үед ажиллаж байгаа процесс бүрийг хоёр болгоно. Эхний `fork()`-ийн дараа 2, хоёр дахийн дараа 4, гурав дахийн дараа 8 процесс болно. Ерөнхийдөө n удаа дараалан `fork()` дуудвал 2^n процесс үүснэ. Тэдгээрийн $2^n - 1 = 7$ нь шинээр үүссэн хүү процесс юм.

Шалгалт: процесс бүр өөрийн PID-г хэвлэхээр өөрчилж ажиллуулахад 8 мөр гарсан.

Дасгал 3.3

Асуулт. Apple-ийн iOS үйлдлийн системийн анхны хувилбарууд зэрэгцээ боловсруулалт (concurrent processing) хийх боломжгүй байсан. Зэрэгцээ боловсруулалт нь үйлдлийн системд нэмдэг гол гурван хүндрэлийг тайлбарла.

Хариулт.

1. **CPU хуваарилалт ба контекст сэлгэлт.** Олон процесс нэг CPU-г хуваан ашиглах тул үйлдлийн систем аль процессыг хэзээ ажиллуулахыг шийдэх процесс хуваарилагч (scheduler)-тэй байх ёстой. Процесс солих бүрт процессын төлөвийг ПУБ (PCB)-д хадгалж, сэргээх шаардлагатай. Энэ нь нэмэлт ачаалал (overhead) үүсгэнэ. Мөн хариу үйлдлийн хурд, шударга байдал, батарейн хэрэглээг тэнцвэржүүлэх хэрэгтэй болно.
2. **Санах ойн удирдлага ба хамгаалалт.** Олон процесс санах ойд нэгэн зэрэг байрлах тул процесс бүрийн хаягийн орныг тусгаарлаж, нэг процесс нөгөөгийнхөө санах ойг эвдэхээс хамгаалах ёстой. Түүнчлэн хязгаарлагдмал санах ойг процессуудын хооронд хуваарилах, дутагдвал аль процессыг зогсоох вэ гэдгийг шийдэх хэрэгтэй.
3. **Нөөцийн зөрчил, синхрончлол.** Хэд хэдэн процесс файл, төхөөрөмж, сүлжээ зэрэг нөөцөд зэрэг хандана. Энэ үед race condition, deadlock үүсэхээс сэргийлэх синхрончлолын механизм (lock, semaphore) болон процесс хоорондын харилцаа (IPC) шаардлагатай болно.

Дасгал 3.4

Асуулт. Зарим компьютерийн систем олон регистрийн багц (register set) агуулдаг. Шинэ контекст аль нэг регистрийн багцад аль хэдийн

ачаалагдсан байхад контекст сэлгэлт (context switch) хийхэд юу болохыг тайлбарла. Шинэ контекст регистрийн багцад биш санах ойд байгаа бөгөөд бүх регистрийн багц ашиглагдаж байвал юу болох вэ?

Хариулт.

- **Шинэ контекст регистрийн багцад байгаа бол** CPU идэвхтэй регистрийн багцыг заасан заагчийг (register set pointer) өөрчлөхөд л хангалттай. Регистрүүдийн утгыг санах ой руу хадгалах, санах ойгоос унших шаардлагагүй тул контекст сэлгэлт маш хурдан, ердөө хэдэн тактад хийгдэнэ.
- **Шинэ контекст санах ойд байгаа бөгөөд бүх багц ашиглагдаж байвал** энгийн контекст сэлгэлт хийгддэг. Эхлээд аль нэг регистрийн багцыг сонгож (жишээ нь хамгийн удаан ашиглагдаагүйг), түүний агуулгыг харгалзах процессын ПУБ буюу санах ой руу хадгална. Дараа нь шинэ процессын контекстыг санах ойгоос тэр багц руу ачаална. Ингэснээр олон регистрийн багцын давуу тал алга болж, контекст сэлгэлтийн хугацаа санах ойн хандалтын хурдаас хамаарна.

Дасгал 3.5

Асуулт. Процесс `fork()` үйлдлээр шинэ процесс үүсгэх үед дараах төлөвүүдийн аль нь эцэг болон хүү процессын хооронд хуваалцагдах вэ? (a) Stack, (b) Heap, (c) Дундын санах ойн сегментүүд (shared memory segments)

Хариулт.

Зөвхөн **(c) дундын санах ойн сегментүүд** хуваалцагдана.

- **Stack** ба **Heap**: `fork()`-ийн дараа хүү процесс эцгийнхээ stack, heap-ийн хуулбарыг авна. Орчин үеийн системд copy-on-write аргаар хуулдаг. Аль нэг нь өөрчлөлт хийвэл тусдаа хуудас үүсэх тул нөгөөд нь харагдахгүй. Иймд эдгээр нь хуваалцагдахгүй.
- **Дундын санах ойн сегмент**: эцэг процесс `fork()`-оос өмнө холбосон (`shm_open` + `mmap`, `MAP_SHARED`) дундын санах ойн сегмент хүү процесст ч мөн адил физик санах ой руу холбогдсон хэвээр үлдэнэ. Иймд нэгнийх нь хийсэн өөрчлөлт нөгөөд нь харагдана.

Дасгал 3.6

Асуулт. RPC механизмын “exactly once” (яг нэг удаа) семантикийг авч үзье. Клиент рүү буцааж илгээсэн ACK мессеж сүлжээний асуудлаас болж алга болсон ч энэ семантикийг хэрэгжүүлэх алгоритм зөв ажиллах уу? Мессежүүдийн дарааллыг тайлбарлаж, “exactly once” хадгалагдах эсэхийг хэлэлц.

Хариулт.

Тийм, алгоритм зөв ажиллаж, **“exactly once” хадгалагдана.**

Сурах бичигт (3.8.2) “exactly once”-г дараах хоёр механизмаар хэрэгжүүлдэг: (1) сервер мессеж бүрийн давтагдашгүй дугаар буюу timestamp-ийн түүхийг хадгалж, давхардсан хүсэлтийг илрүүлнэ; (2) клиент ACK хүлээн авах хүртэл хүсэлтээ тодорхой хугацааны давтамжтайгаар дахин илгээнэ.

ACK алга болох үеийн мессежийн дараалал:

1. Клиент `RPC(id=42)` хүсэлтийг илгээнэ.
2. Сервер хүсэлтийг хүлээн авч, `id=42` түүхэнд байхгүй тул процедурыг ажиллуулна. `id=42`-ийг үр дүнгийн хамт түүхэнд бичиж, `ACK(42)` илгээнэ.
3. `ACK(42)` сүлжээнд алга болно.
4. Клиентийн хүлээх хугацаа (timeout) дуусахад тэр ижил `RPC(id=42)`-г дахин илгээнэ.
5. Сервер `id=42`-ийг түүхээсээ олж, хүсэлтийг давхардсан гэж үзнэ. Процедурыг дахин ажиллуулахгүй, зөвхөн хадгалсан үр дүн бүхий `ACK(42)`-ийг дахин илгээнэ.
6. Клиент `ACK(42)`-ийг хүлээн авч дахин илгээхээ зогсооно.

Процедур серверт яг нэг удаа ажилласан бөгөөд клиент үр дүнгээ авсан. Иймд ACK алга болох нь зөвхөн дахин илгээлт үүсгэх боловч давхар гүйцэтгэл үүсгэхгүй, “exactly once” хадгалагдана.

Дасгал 3.7

Асуулт. Тархсан систем серверийн доголдолд (server failure) өртөмтгий гэж үзье. RPC-ийн “exactly once” семантикийг баталгаажуулахын тулд ямар механизм шаардлагатай вэ?

Хариулт.

Сервер унах үед санах ойд хадгалсан түүх устах тул дахин асаад ирсэн сервер өмнө нь аль хүсэлтийг гүйцэтгэснээ мэдэхгүй болно. Иймд дараах механизмууд шаардлагатай:

1. **Тогтвортой санах ой дахь лог (stable storage / write-ahead log).** Сервер хүлээн авсан хүсэлтийн ID (timestamp), гүйцэтгэлийн төлөв, үр дүнг ACK илгээхээс өмнө дискэн дээрх логт бичнэ. Дахин асахдаа логоо уншиж давхардсан хүсэлтийг илрүүлнэ.
2. **Атомар гүйцэтгэл (transaction).** Процедурын гүйцэтгэл болон “гүйцэтгэсэн” гэсэн бичлэгийг нэг атомар үйлдэл болгоно. Commit/roll-back ашигласнаар хагас гүйцэтгэгдсэн төлөвт үлдэхгүй.

3. **Давтагдашгүй хүсэлтийн ID.** Клиент хүсэлт бүрт давтагдашгүй дугаар (жишээ нь клиентийн ID ба дарааллын дугаар) өгнө. Сервер дахин асах бүрт шинэ “epoch” дугаар ашиглаж, хуучин болон шинэ хүсэлтийг ялгана.

4. **Клиент талын timeout ба давтан илгээлт.** Клиент серверийн хариуг авах хүртэл хүсэлтээ ижил ID-тайгаар дахин илгээнэ. Сервер логоос шалгаж, гүйцэтгэсэн бол хадгалсан үр дүнг, гүйцэтгээгүй бол шинээр гүйцэтгэсэн үр дүнг буцаана.

Эдгээрийг хослуулснаар сервер унаж босох тохиолдолд ч хүсэлт бүр яг нэг удаа гүйцэтгэгдэх баталгаатай болно.